Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 7

Дисциплина МДК 02.02
Диаграммы модулей и структура решения

Вариант № 2
АИС «Электронная медицинская карта»

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Состав Solution.....3

1.2 Диаграммы компонентов.....4

1.3 Каталоги и точки запуска.....5

2 ПРОВЕРКА РЕЗУЛЬТАТОВ.....6

ЗАКЛЮЧЕНИЕ.....7

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....8

ПРИЛОЖЕНИЕ А.....9

А.1 Листинг Models.....9

1 ВЫПОЛНЕНИЕ РАБОТЫ

Цель работы — распределить логику между проектами Core, Infrastructure, UI и Tests; построить UML Component и C4 Level 3 и проверить зависимости. Работа выполнена по варианту 2 нового сборника [2].

В примерах используются вымышленные пациенты. Оформление выполнено по требованиям института [1].

1.1 Состав Solution

Сведения представлены в таблице 1.

Таблица 1 — Проекты решения Emr.sln

Проект	Назначение	Прямые зависимости
AIS.Core	Модели, правила и интерфейсы	Нет
AIS.Infrastructure	SQLite, HTTP и AES-GCM	Core
AIS.Presentation	ViewModel / ICommand	Core
AIS.UI	WPF View	Core, Presentation
AIS.Desktop	Сборка desktop-клиента	UI, Infrastructure, Presentation
AIS.Gateway	API и проверочный интерфейс	Core, Infrastructure, Presentation
AIS.MappingCli	Импорт карточки	Core, Infrastructure
AIS.Tests	Сетевые и модульные проверки	Core, Infrastructure, Presentation, Gateway

Источник: составлено по учебному заданию [2].

ViewModel выделена в переносимую библиотеку, чтобы тестировать её без Windows. Desktop и Gateway — точки сборки зависимостей: они знают об адаптерах, а Core остаётся независимым. Такое разделение соответствует направлению зависимостей к ядру [4].

1.2 Диаграммы компонентов

Результат показан на рисунке 1.

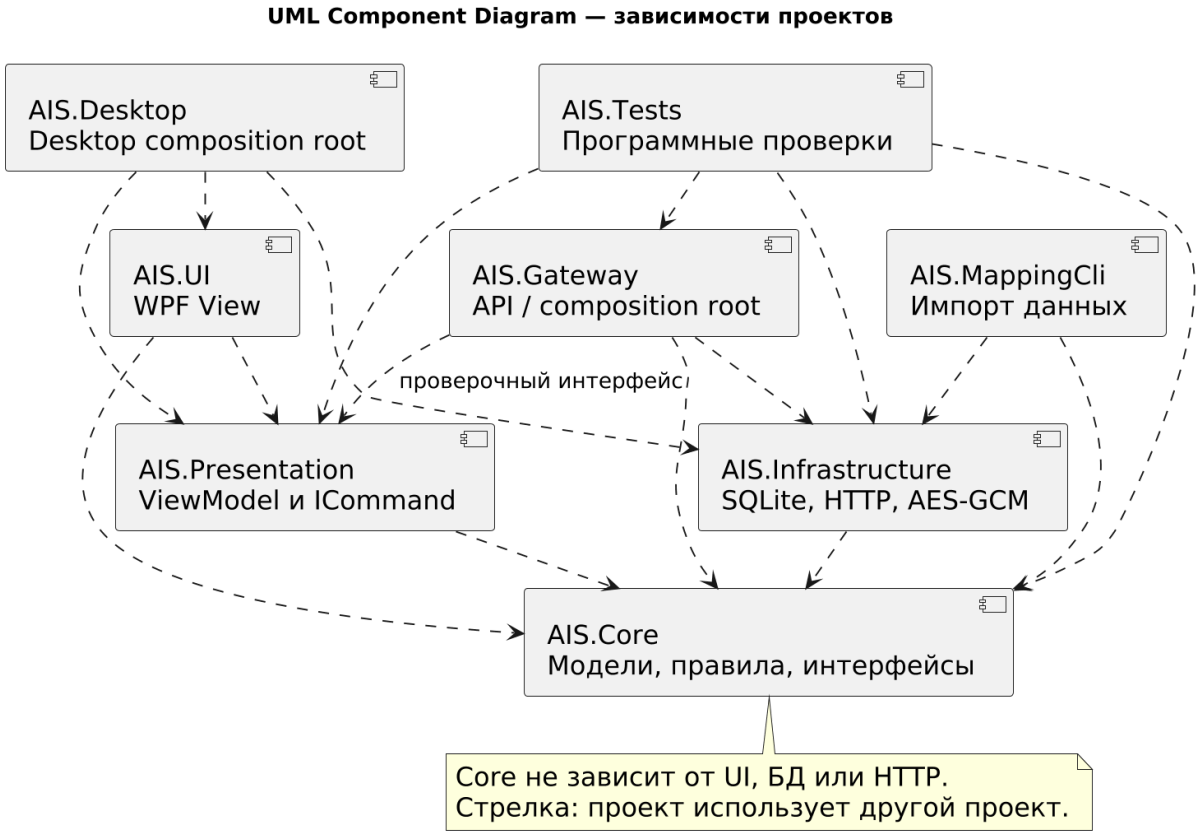


Рисунок 1 — UML Component Diagram проектов решения

Источник: схема разработана для учебного проекта.

Стрелка зависимости направлена от использующего проекта к используемому. Связь Gateway → Presentation нужна только проверочному интерфейсу ViewModel; производственный обработчик доступа использует правила Core.

Результат показан на рисунке 2.

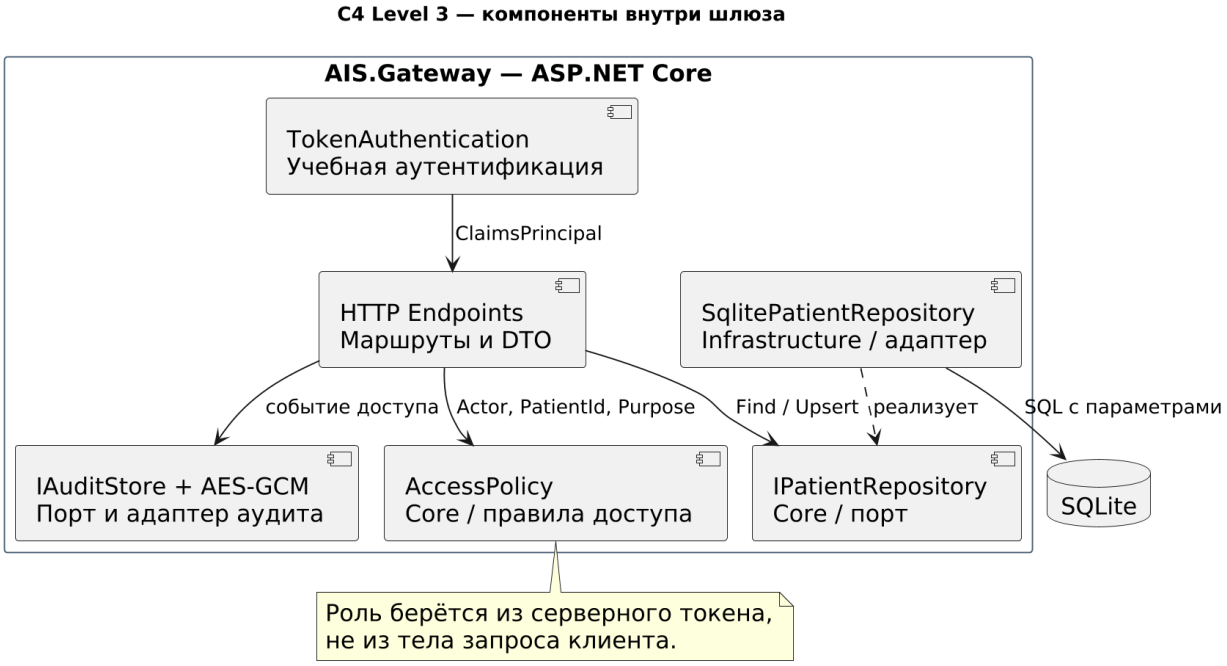


Рисунок 2 — C4 Level 3 — компоненты интеграционного шлюза

Источник: схема разработана для учебного проекта.

C4 Level 3 раскрывает компоненты внутри выбранного контейнера, а не повторяет контекст всей организации [3]. На схеме показаны порты Core и адаптеры Infrastructure.

1.3 Каталоги и точки запуска

```
Emr.sln
src/AIS.Core/{Models,PatientMapper,AccessPolicy}.cs
src/AIS.Infrastructure/
  SqlitePatientRepository.cs
  EncryptedAuditStore.cs
  MedicalApiClient.cs
src/AIS.Presentation/PatientViewModel.cs
src/AIS.UI/PatientWindow.cs
src/AIS.Desktop/Program.cs
src/AIS.Gateway/{GatewayFactory,TokenAuthentication}.cs
src/AIS.MappingCli/Program.cs
tests/AIS.Tests/Program.cs
```

Архив ПР 7 содержит решение Emr.sln и исходный код всех проектов. Сборка запускается командой dotnet build Emr.sln.

2 ПРОВЕРКА РЕЗУЛЬТАТОВ

Программно проверены наличие восьми проектов, отсутствие ProjectReference в Core, отсутствие зависимости UI → Infrastructure, существование ссылок и отсутствие циклов. Результаты сохранены в architecture.json и общем журнале тестов.

Сведения представлены в таблице 2.

Таблица 2 — Контроль модульной структуры

Критерий	Результат
Core независим	PASS
UI не использует Infrastructure напрямую	PASS
Ссылки проектов разрешаются	PASS
Граф не содержит циклов	PASS
Все проекты компилируются	8 из 8

Источник: составлено по учебному заданию [2].

Интерфейсы ядра приведены в приложении А. Полные реализации и тесты поставляются в отдельном архиве программы ПР 7.

Исходники и материалы актуальной публикации: https://github.com/rob4ik72-del/MDK0202-EMR

ЗАКЛЮЧЕНИЕ

Построены UML Component и C4 Level 3, создано решение из восьми C#-проектов. Структура на схемах сверена с фактическими ProjectReference.

Реализации хранилища и HTTP можно менять без переноса правил доступа в UI. Циклических зависимостей не обнаружено.

Репозиторий проекта: https://github.com/rob4ik72-del/MDK0202-EMR

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. СПбИЭУ. Положение о требованиях по подготовке и оформлению письменных работ и приложение М: локальный нормативный акт, предоставленный в материалах обучения. [1].

2. МДК 02.02. Детализированный сборник практических работ ПР1–ПР9: электронный учебный материал; вариант 2 «Электронная медицинская карта». [2].

3. Simon Brown. The C4 model: diagrams. URL: https://c4model.com/diagrams (дата обращения: 05.11.2026) [3].

4. Microsoft Learn. Common web application architectures. URL: https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures (дата обращения: 05.11.2026) [4].

ПРИЛОЖЕНИЕ А

А.1 Листинг Models

src/AIS.Core/Models.cs

```
namespace Ais.Core;
public sealed record SourcePatient(string ExternalId, string FullName, string BirthDate, string Sex, string
PolicyNumber, string UpdatedAt);
public sealed record Patient(int ExternalId, Guid Id, string FullName, DateOnly BirthDate, string Sex, string
PolicyNumber, DateTimeOffset UpdatedAt);
public sealed record PatientSummary(Guid Id, string FullName, string BirthDate, string Sex, string Note);
public sealed record AccessRequest(Guid RequestId, Guid PatientId, string Purpose, string WorkstationId);
public sealed record AccessReply(Guid RequestId, Guid PatientId, string Decision, string Scope);
public sealed record ApiError(string Code, string Message, Dictionary<string, string[]>? Errors = null);
public sealed record Actor(string Id, string Role, Guid[] AssignedPatients);
public interface IPatientRepository
{
   Patient? Find(Guid id);
   string Upsert(Patient patient);
}

public interface IAuditStore
{
   void Write(string actor, Guid patient, string action, string result);
}

public interface IMedicalApiClient
{
   Task<PatientSummary> GetSummaryAsync(Guid id, CancellationToken ct = default);
   Task<AccessReply> CheckAccessAsync(AccessRequest request, CancellationToken ct = default);
}

public static class SampleData
{
   public static readonly Guid PatientId = Guid.Parse("11111111-1111-4111-8111-111111111111");
   public static Patient Patient => new(42, PatientId, "Пациент Учебный Один", new DateOnly(1990, 2, 14),
"F", "0012345678901234", DateTimeOffset.Parse("2026-10-20T10:00:00+03:00"));
   public static SourcePatient Source => new("000042", "  Пациент   Учебный Один ", "14.02.1990", "Ж",
"0012345678901234", "2026-10-20T10:00:00+03:00");
}
```